

Functional and compositional approaches to scalable statistical computing

Darren Wilkinson

Durham University, UK

darrenjw.github.io

Compositional Foundations of Statistics & Machine learning

Warwick University, UK, July 2026

Overview

Background

Bayesian computation

Category Theory

Probability monads

 Implementations of probability monads

Comonads, recursion schemes and optics

Conclusion

Background

Background

- For non-trivial problems, Bayesian computation typically relies on computationally intensive methods, such as MCMC, SMC or ABC
- These often require a custom implementation in some programming language
- All of the languages commonly used are very old, dating back to the dawn of the computing age, and are quite unsuitable for scalable and efficient Bayesian computation
- Interpreted dynamic languages such as R, Python and Matlab are far too slow, among many other things
- Languages such as C, C++, Java and Fortran are much faster, but are also very poorly suited to the development of efficient, well-tested, compositional, scalable code, able to take advantage of modern computing hardware

Alternative languages and approaches

- All of the languages on the previous slide are fundamentally **imperative** programming languages, mimicking closely the way computer processors actually operate
- There have been huge advances in computing science in the decades since these languages were created, and many new, different and better programming languages have been created
- Although **functional** programming (FP) languages are not new, there has been a large resurgence of interest in functional languages in the last decade or two, as people have begun to appreciate the advantages of the functional approach, especially in the context of developing large, scalable software systems, and the ability to take advantage of modern computing hardware
- There has also been a swing away from **dynamically typed** programming languages back to **statically typed** languages

Functional programming

- FP languages emphasise the use of **immutable** data, **pure**, **referentially transparent functions**, and **higher order functions**
- FP languages more naturally support composition of models, data and computation than imperative languages, leading to more scalable and testable software
- Statically typed FP languages (such as Haskell and Scala) correspond closely to the **simply-typed lambda calculus** which is one of the canonical examples of a **Cartesian closed category** (CCC)
- This connection between typed FP languages and CCCs enables the borrowing of ideas from category theory into FP
- Category theory concepts such as **functors**, **monads** and **comonads** are useful for simplifying code that would otherwise be somewhat cumbersome to express in pure FP languages

Concurrency, parallelism, distribution, state

- Modern computing platforms feature processors with many cores, and possibly many such processors — parallel programming is required to properly exploit these
- Most of the notorious difficulties associated with parallel programming revolve around **shared mutable state**
- In pure FP, state is not mutable, so there is no mutable state, shared or otherwise
- Consequently, most of the difficulties typically associated with parallel, distributed, and concurrent programming simply don't exist in FP — parallelism in FP is so easy and natural that it is sometimes completely automatic
- This natural scalability of FP languages is one reason for their recent resurgence

Compositionality

- Not all issues relating to scalability of models and algorithms relate to parallelism
- A good way to build a large model is to construct it from smaller models
- A good way to develop a complex computation is to construct it from simpler computations
- This (recursive) decomposition-composition approach is at the heart of the so-called “divide and conquer” approach to problem solution, and is very natural in FP (eg. FFT and BP for PGMs)
- It also makes code much easier to **test** for correct behaviour
- Category theory is in many ways the mathematical study of (associative) composition, and this leads to useful insights

Bayesian computation

Bayesian computation

- **map-reduce** operations on **functorial** data collections can trivially parallelise (and distribute):
 - Likelihood evaluations for big data
 - ABC algorithms
 - SMC re-weighting and re-sampling
- Gibbs sampling algorithms can be implemented as **cobind** operations on an appropriately coloured (parallel) **comonadic** conditional independence graph
- **Probabilistic programming languages** (PPLs) can be implemented as embedded domain specific languages (DSLs) trivially using **for/do** syntax for **monadic composition** in conjunction with **probability monads**
- **Automatic differentiation** (AD) is natural and convenient in functional languages, facilitating gradient-based algorithms such as MALA, HMC and PDMP

Parallel immutable collections

- Using `map` to apply a `pure` function to all of the elements in a functorial collection can clearly be done in parallel
- So if the collection contains n elements, then the computation time can be reduced from $O(n)$ to $O(1)$ (on infinite parallel hardware)
 - `Vector(3,5,7) map (_ * 2) = Vector(6,10,14)`
- We can carry out `reductions` as `folds` over collections:
`Vector(6,10,14) reduce (_ + _) = 30`
- In general, sequential folds can not be parallelised, but...

Monoids and parallel “map–reduce”

- A **monoid** is a very important concept in FP
- For now we will think of a monoid as a **set** of elements with a **binary relation** $\star$ which is **closed** and **associative**, and having an **identity** element wrt the binary relation
- You can think of it as a **semi-group** with an identity or a **group** without an inverse
- **folds**, **scans** and **reduce** operations can be computed in parallel using **tree reduction**, reducing time from $O(n)$ to $O(\log n)$ (on infinite parallel hardware)
- “**map–reduce**” is just the pattern of processing large amounts of data in an immutable collection by first **mapping** the data (in parallel) into a monoid and then **tree-reducing** the result (in parallel), sometimes called **foldMap**

Distributed parallel collections with Apache Spark

- **Apache Spark** (`spark.apache.org`) is a Scala library for distributed Big Data processing on (large) clusters of machines
- The basic datatype provided by Spark is an **RDD** — a resilient distributed dataset
- An RDD is just a **lazy, distributed**, parallel monadic collection, supporting methods such as `map`, `flatMap`, `reduce`, etc., which can be used in exactly the same way as any other Scala collection
- Code looks exactly the same whether the RDD is a small dataset on a laptop or terabytes in size, distributed over a large Spark cluster
- It is a powerful framework for the development of scalable algorithms for Bayesian computation

- **JAX** (jax.readthedocs.io) is a Python library embedding a DSL for automatic differentiation and JIT-compiling (array) functions to run very fast on (multiple) CPU or GPU
- It is especially good at speeding up likelihood evaluations and (MCMC-based) sampling algorithms for complex models
- It is not unheard of for MCMC algorithms to run 100 times faster than regular Python code, even on a single CPU (multiple cores and GPUs will speed things up further)
- The JAX eDSL is **pure functional array language**
- Despite targeting a completely different kind of scalability to Spark, and being embedded in a very different language, the fundamental computational model is very similar: **express algorithms in terms of lazy transformations of immutable data structures using pure functions**

Functional algorithms

- By expressing algorithms in a functional style (eg. lazy transformations of immutable data structures with pure functions), we allow many code optimisations to be automatically applied
- Pure functional algorithms are relatively easy to analyse, optimise, compile, parallelise, distribute, differentiate, push to GPU, etc.
- These optimisations can typically be done automatically by the library, compiler, framework, etc., without significant user intervention
- It is very difficult (often impossible) to analyse and reason about imperative code in a similar way

- (Unnormalised) log-posteriors expressed in JAX can be sampled using a variety of different algorithms (including HMC and NUTS, via auto-diff) using **BlackJAX** (blackjax-devs.github.io/blackjax/)
- Python PPLs such as **PyMC** and **NumPyro** (can) use JAX on the backend
- Neural network libraries such as **Keras** and **Flax** (can) use JAX as the backend
- Since JAX is generally better suited to scientific computing than the other popular ML frameworks, there is a growing scientific computing ecosystem based on JAX

Category Theory

Category theory

- A category $\mathcal{C}$ consists of a collection of **objects**, $\text{ob}(\mathcal{C})$, and **morphisms**, $\text{hom}(\mathcal{C})$. Each morphism is an ordered pair of objects (an arrow between objects). For $x, y \in \text{ob}(\mathcal{C})$, the set of morphisms from x to y is denoted $\text{hom}_{\mathcal{C}}(x, y)$.
 $f \in \text{hom}_{\mathcal{C}}(x, y)$ is often written $f : x \longrightarrow y$.
- Morphisms are closed under **composition**, so that if $f : x \longrightarrow y$ and $g : y \longrightarrow z$, then there must also exist a morphism $h : x \longrightarrow z$ written $h = g \circ f$.
- Composition is associative, so that $f \circ (g \circ h) = (f \circ g) \circ h$ for all composable $f, g, h \in \text{hom}(\mathcal{C})$.
- For every $x \in \text{ob}(\mathcal{C})$ there exists an **identity** morphism $\text{id}_x : x \longrightarrow x$, with the property that for any $f : x \longrightarrow y$ we have $f = f \circ \text{id}_x = \text{id}_y \circ f$.

Functors

- A **functor** is a mapping from one category to another which preserves some structure
- A functor F from $\mathcal{C}$ to $\mathcal{D}$, written $F : \mathcal{C} \longrightarrow \mathcal{D}$ is a pair of functions (both denoted F):
 - $F : \text{ob}(\mathcal{C}) \longrightarrow \text{ob}(\mathcal{D})$
 - $F : \text{hom}(\mathcal{C}) \longrightarrow \text{hom}(\mathcal{D})$, where $\forall f \in \text{hom}(\mathcal{C})$, we have $F(f : x \longrightarrow y) : F(x) \longrightarrow F(y)$
 - In other words, if $f \in \text{hom}_{\mathcal{C}}(x, y)$, then $F(f) \in \text{hom}_{\mathcal{D}}(F(x), F(y))$
- The functor must satisfy the **functor laws**:
 - $F(\text{id}_x) = \text{id}_{F(x)}, \forall x \in \text{ob}(\mathcal{C})$
 - $F(f \circ g) = F(f) \circ F(g)$ for all composable $f, g \in \text{hom}(\mathcal{C})$
- A functor $F : \mathcal{C} \longrightarrow \mathcal{C}$ is called an **endofunctor** — in the context of functional programming, the word functor usually refers to an endofunctor $F : \mathbf{Set} \longrightarrow \mathbf{Set}$

Laziness, composition, laws and optimisations

- Laziness allows some optimisations to be performed that would be difficult to automate otherwise
- Consider a dataset `rdd: RDD[T]`, functions `f: T => U`, `g: U => V`, and a binary operation `op: (V,V) => V` for monoidal type `V`
- We can map the two functions and then reduce with:
 - `rdd map f map g reduce op`
 - to get a value of type `V`, all computed in parallel
- However, re-writing this as:
 - `rdd map (g compose f) reduce op`
 - would eliminate an intermediate collection, but is equivalent due to the 2nd functor law (`map fusion`)
- Category theory **laws** often correspond to **optimisations** that can be applied to code without affecting results — Spark can do these optimisations **automatically** due to lazy evaluation

Natural transformations

- Often there are multiple functors between pairs of categories, and sometimes it is useful to be able to transform one to another
- Suppose we have two functors $F, G : \mathcal{C} \longrightarrow \mathcal{D}$
- A **natural transformation** $\alpha : F \Rightarrow G$ is a family of morphisms in $\mathcal{D}$, where $\forall x \in \mathcal{C}$, the **component** $\alpha_x : F(x) \longrightarrow G(x)$ is a morphism in $\mathcal{D}$
- To be considered **natural**, this family of morphisms must satisfy the **naturality law**:
 - $\alpha_y \circ F(f) = G(f) \circ \alpha_x, \quad \forall f : x \longrightarrow y \in \text{hom}(\mathcal{C})$
- **Naturality** is one of the most fundamental concepts in category theory
- In the context of FP, a natural transformation could (say) map an **Option** to a **List** (with at most one element)

Monads

- A **monad** on a category $\mathcal{C}$ is an endofunctor $T : \mathcal{C} \longrightarrow \mathcal{C}$ together with two natural transformations $\eta : \text{Id}_{\mathcal{C}} \longrightarrow T$ (**unit**) and $\mu : T^2 \longrightarrow T$ (**multiplication**) fulfilling the **monad laws**:
 - **Associativity**: $\mu \circ T\mu = \mu \circ \mu_T$, as transformations $T^3 \longrightarrow T$
 - **Identity**: $\mu \circ T\eta = \mu \circ \eta_T = 1_T$, as transformations $T \longrightarrow T$
- The associativity law says that the two ways of **flattening** $T(T(T(x)))$ to $T(x)$ are the same
- The identity law says that the two ways of **lifting** $T(x)$ to $T(T(x))$ and then flattening back to $T(x)$ both get back to the original $T(x)$
- In FP, we often use **M** (for monad) rather than T (for triple), and say that there are three monad laws — the identity law is considered to be two separate laws

Kleisli category

- Kleisli categories formalise monadic composition
- For any monad T over a category $\mathcal{C}$, the **Kleisli category** of $\mathcal{C}$, written $\mathcal{C}_T$ is a category with the same objects as $\mathcal{C}$, but with morphisms given by:
 - $\text{hom}_{\mathcal{C}_T}(x, y) = \text{hom}_{\mathcal{C}}(x, T(y)), \forall x, y \in \text{ob}(\mathcal{C})$
- The identity morphisms in $\mathcal{C}_T$ are given by $\text{id}_x = \eta(x), \forall x$, and morphisms $f : x \longrightarrow T(y)$ and $g : y \longrightarrow T(z)$ in $\mathcal{C}$ can compose to form $g \circ_T f : x \longrightarrow T(z)$ via
 - $g \circ_T f = \mu_z \circ T(g) \circ f$leading to composition of morphisms in $\mathcal{C}_T$.
- In FP, the morphisms in $\mathcal{C}_T$ are often referred to as **Kleisli arrows**, or **Kleislis**, or sometimes just **arrows** (although **Arrow** usually refers to a generalisation of Kleisli arrows, sometimes known as **Hughes arrows**)

Comonads

- The comonad is the categorical dual of the monad, obtained by “reversing the arrows” for the definition of a monad
- A **comonad** on a category $\mathcal{C}$ is an endofunctor $W : \mathcal{C} \rightarrow \mathcal{C}$ together with two natural transformations $\varepsilon : W \rightarrow \text{Id}_{\mathcal{C}}$ (**counit**) and $\delta : W \rightarrow W^2$ (**comultiplication**) fulfilling the **comonad laws**:
 - **Associativity**: $\delta_W \circ \delta = W\delta \circ \delta$, as transformations $W \rightarrow W^3$
 - **Identity**: $\varepsilon_W \circ \delta = W\varepsilon \circ \delta = 1_W$, as transformations $W \rightarrow W$
- The associativity law says that the two ways of **duplicating** a $W(x)$ duplicated to a $W(W(x))$ to a $W(W(W(x)))$ are the same
- The identity law says that the two ways of **extracting** a $W(x)$ from a $W(x)$ duplicated to a $W(W(x))$ are the same

Typeclasses for Monoid, Functor, Monad and Comonad

```
trait Monoid[A] {  
  def combine(a1: A, a2: A): A  
  def id: A  
}  
  
trait Functor[F[_]] {  
  def map[A,B](fa: F[A])(f: A => B): F[B]  
}  
  
trait Monad[M[_]] extends Functor[M] {  
  def pure[A](a: A): M[A]  
  def flatMap[A,B](ma: M[A])(f: A => M[B]): M[B]  
}  
  
trait Comonad[W[_]] extends Functor[W] {  
  def extract[A](wa: W[A]): A  
  def coflatMap[A,B](wa: W[A])(f: W[A] => B): W[B]  
}
```

Scalable ABC

- Simulate a collection of parameters, `pars: C[Param]`, from a prior and store in a generic (functor) collection
- Define essential features of the ABC model:

```
realData: Data
simModel: Param => Data
summ: Data => SS
dist: (SS, SS) => Double
```

- Then scalable (parallelism-agnostic) ABC is as simple as:

```
val realSS = summ(realData)
pars.map(summ compose simModel).
  filter(dist(_, realSS) < eps)
```

using map-fusion to avoid constructing a collection of synthetic datasets

Probability monads

Composing random variables with the probability monad

- The **probability monad** provides a foundation for describing random variables in a pure functional way (cf. **Giry monad**)
- We can build up joint distributions from marginal and conditional distributions using **monadic composition**
- For example, consider an exponential mixture of Poissons (marginally negative binomial): we can think of an exponential distribution parametrised by a rate as a function `Exponential: Double => Rand[Double]` and a Poisson parametrised by its mean as a function `Poisson: Double => Rand[Int]`
- Those two functions don't directly compose, but do in the Kleisli category of the `Rand` monad, so `Exponential(3) flatMap {Poisson(_)}` will return a `Rand[Int]` which we can draw samples from if required

Monads for probabilistic programming

- For larger probability models we can use **for-comprehensions** to simplify the model building process, eg.

```
for { mu <- Gaussian(10,1)
      tau <- Gamma(1,1)
      sig = 1.0/sqrt(tau)
      obs <- Gaussian(mu,sig) }
yield ((mu,tau,obs))
```

- We can use a regular probability monad for building forward models this way, and even for building models with simple Bayesian inference procedures allowing conditioning
- For sophisticated probabilistic sampling algorithms (eg. SMC, MCMC, pMCMC, HMC, ...) and hybrid compositions, it is better to build models like this using a **free monad** which can be **interpreted** in different ways

Probability monad foundations

Mathematically, what **exactly** is a probability monad P ?

Standard answer:

- **Giry monad** – measurable functions and spaces
 - Defined on the category **Meas** of measurable spaces, P sends X to the space of probability measures on X
 - η is a dirac measure ($\eta(x) = \delta_x$) and μ is defined as marginalisation using Lebesgue integration

$$\mu_X(\rho)(A) = \int_{P(X)} \tau_A(d\rho)$$

- Provides a solid foundation matching up closely with conventional probability theory, but isn't as compositional as we'd like (eg. **Meas** is not cartesian closed)
- Awkward for (higher-order) probabilistic programming languages

Alternative probability monad foundations

- **Quasi-Borel spaces**
 - A modification of the measure space approach which is cartesian closed (eg. $\mathbb{R}^{\mathbb{R}} = \mathbf{QBS}(\mathbb{R}, \mathbb{R})$)
 - Good for the denotational semantics of (higher-order) probabilistic programming languages where we want to define probability distributions over functions
- **Kantorovich monad** – built on (complete) metric spaces
 - Provides an alternative, more composable foundation for probability theory, less tightly linked to measure theory
- **Expectation monad** – directly define an expectation monad
 - A formulation in which expectation is primitive and probability is a derived concept (cf. de Finetti)

And several others...

Probability monads for uncertainty quantification (UQ)

- Suppose we have a **computer model**, f , mapping inputs (or parameters) $x \in X$ to output $y \in Y$:

$$f : X \longrightarrow Y$$

- Suppose further that we are uncertain about the “correct” inputs to use, and encapsulate our uncertainty in a probability distribution $p_X \in P(X)$, where P denotes a probability monad
- We can’t apply f to p_X (the types don’t match), but we can use the fact that P is a functor to “lift” f into the monad (`px map f`):

$$P(f) : P(X) \longrightarrow P(Y),$$

inducing a probability distribution on the output.

UQ for stochastic computer models

- Suppose now that we have a stochastic (non-deterministic) computer model (or emulator):

$$g : X \longrightarrow P(Y),$$

so that for an given fixed input $x \in X$, the output is a probability distribution over Y

- If we have additional uncertainty for the input $x \in X$, then we lift g into the Kleisli category of the monad (`px flatMap g`):

$$\mu_Y \circ P(g) : P(X) \longrightarrow P(Y),$$

effectively just marginalising over our uncertainty about x

Rand for forward simulation

- Whilst the semantics of probability monads should be reasonably clear, there are many different ways to implement them, depending on the intended use-case
- The simplest probability monad implementations typically provide a `draw` method, which can be used (with a uniform random number generator) to generate draws from the distribution of interest
- For `x: Rand[X]`, monadic bind `x flatMap (f: X => Rand[Y])` returns `y: Rand[Y]`
- `f` represents a conditional distribution and `y` represents the marginalisation of this distribution over `x`
- The `draw` method for `y` first calls `draw` on `x` (which it holds a reference to), feeds this in to `f` and then calls `draw` on the result

Dist for Bayesian conditioning via SMC

- Rather than providing a `draw` method for generating individual values from the distribution of interest, you can define a monad whose values represent large (weighted) random samples from a distribution — an empirical distribution
- `flatMap` is then essentially just the same `flatMap` you would have on any other collection, but here will typically be combined with a random thinning of the result set to prevent an explosion in the number of particles with deep chaining
- One advantage of this representation is that it then easy to introduction a `condition` method which uses importance (re)sampling to condition on observations
- This can be used to implement a simple SMC-based Bayesian PPL with very little code, but it won't scale well with large or complex models

RandomVariable for Bayesian HMC sampling

- Rather than using a probability monad to represent samples or methods for sampling, one can instead use them to represent the (joint, log) density of the variables
- `flatMap` just multiplies the (conditional) densities
- Again, conditioning is easy (multiplication), so this forms a good basis for Bayesian PPLs
- Can use the joint posterior for simple MH algorithms (and Gibbs, if dependencies are tracked), but for Langevin and HMC algorithms, also need to keep track of gradients, using automatic differentiation (AD)
- OK, because (reverse-mode) AD on a compute graph is also computationally tractable using categorical constructs

Composing probabilistic programs

- Describing probabilistic programs as **monadic values** in a functional programming language with syntax for monadic composition leads immediately to an **embedded PPL DSL** “for free”
- This in turn enables a fully **compositional** approach to the (scalable) development of (hierarchical) probabilistic models
- Model components can be easily “re-used” in order to build big models from small
- eg. a regression model component can be re-used to create a hierarchical random effects model over related regressions
- In some well-known PPLs, this would require manual copy-and-pasting of code, wrapping with a “for loop”, and manual hacking in of an extra array index into all array references — not at all compositional!

Representation independence using the free monad

- However you implement your probability monad, the semantics of your probabilistic program are (essentially) the same
- It would be nice to be able to define and compose probabilistic programs independently of concerns about implementation, and then to **interpret** the program with a particular implementation later
- Building a probability monad on top of the **free monad** allows this — implementation of **pure** and **flatMap** is “suspended” in a way that allows subsequent interpretation with concrete implementations later
- This allows **layering** of multiple inference algorithms, and different interpretation of different parts of the model, enabling various optimisations and sophisticated **composition** of different (hybrid) inference algorithms

Compositionality of inference algorithms

- As well as building **models** in scalable, compositional way, we would also like our **inference algorithms** to be compositional, ideally reflecting the compositional structure of our models
- Some algorithms, such as component-wise samplers and message-passing algorithms, naturally reflect the compositional structure of the underlying model
- Even “global” algorithms, such as Langevin and HMC samplers, rely on gradient computations that often have a compositional structure
- Understanding and exploiting the **compositional structure** of models and algorithms will be crucial for developing scalable inferential methods

Comonads, recursion schemes and optics

Comonads for statistical computation

- Monads are good for sequencing operations that can be carried out on data points independently
- For computations requiring knowledge of some kind of local neighbourhood structure, **Comonads** are a better fit
- `coflatMap` will take a function representing a **local computation** producing one value for the new structure, and then extend this to generate all values associated with the comonad
- Useful for defining linear filters (including discrete Laplacians), Gibbs samplers, convolutional neural networks, etc.
- Concepts from category theory are often useful for providing abstractions which make convenient computations that might otherwise seem cumbersome within a pure functional programming language

Linear filters for Streams and Images

Let's start with writing a function to compute the weighted average of values at the start of an infinite data `Stream`

```
def linFilter(weights: Stream[Double])(  
  s: Stream[Double]): Double =  
  (weights, s).parMapN(_*_).sum
```

We can extend this `local computation` to the entire `Stream` using `coflatMap`

```
s.coflatMap(linFilter(Stream(0.25,0.5,0.25)))
```

resulting in a new infinite `Stream` containing the filtered values

- Note that the method `extract` will return the value at the head of the `Stream`

Filtering an Image

- For a 2D Image, the relevant context for local computation is less clear, so to become comonadic, the Image class needs to be “pointed”, by augmenting it with a “cursor” pointing to the current pixel of interest
- For the pointed image class, `extract` returns the value of the pixel at the cursor
- We filter using functions which compute a value on the basis of the “neighbourhood” of the cursor
- `map` and `coflatMap` operations will automatically parallelise if the image is backed with a parallel data structure

```
def fil(pi: PImage[Double]):Double = (2*pi.extract+
  pi.up.extract+pi.down.extract+
  pi.left.extract+pi.right.extract)/6.0
val filteredImage = pim0.coflatMap(fil)
def pims = Stream.iterate(pim0)(_ .coflatMap(fil))
```

Gibbs sampling (an Ising model)

```
def gibbsKernel(pi: PImage[Int]): Int = {  
    val sum = pi.up.extract+pi.down.extract+  
        pi.left.extract+pi.right.extract  
    val p1 = math.exp(beta*sum)  
    val p2 = math.exp(-beta*sum)  
    val probplus = p1/(p1+p2)  
    if (new Bernoulli(probplus).draw) 1 else -1  
}  
  
def oddKernel(pi: PImage[Int]): Int =  
    if ((pi.x+pi.y) % 2 != 0) pi.extract else  
        gibbsKernel(pi)  
  
def evenKernel(pi: PImage[Int]): Int =  
    if ((pi.x+pi.y) % 2 == 0) pi.extract else  
        gibbsKernel(pi)  
  
def pims=Stream.iterate(pim0)(_ coflatMap(oddKernel).  
    coflatMap(evenKernel))
```

Recursion schemes

- A **fold** over a sequential data structure is an example of a recursion scheme known more formally as a **catamorphism** (eg. sums and AD)
- The generation of a sequential stream of values from a seed and a generation function (eg. the generation of an MCMC chain) is an example of an **unfold** operation, more formally known as an **anamorphism**
- Generating and then consuming a data structure (eg. simulating an MCMC chain and then computing summary statistics from the iterations) is an example of a **refold**, known formally as a **hylomorphism**

(glossing over **F-algebras**, **coalgebras**, **ADTs** and **fixed point types**)

Again, category theory provides the theory and FP languages provide frameworks and abstractions for implementation

Lenses (and other optics)

- An instance of a simple **lens** of type `Lens[S, A]` is just a pair of functions: `get: S => A` and `set: (S, A) => S`
- In the context of functional programming, `S` is typically a product type, assumed to contain values of type `A`
- Lenses and other related optics are typically used to query and update immutable algebraic data types (ADTs)
- Crucially, lenses **compose** (associatively): a `Lens[S, A]` and a `Lens[A, B]` can be composed to get a `Lens[S, B]`
- Given `l1: Lens[S, A]` and `l2: Lens[A, B]` we can construct `l3: Lens[S, B]` with `l3.get(s) = l2.get(l1.get(s))` and `l3.set(s, a) = l1.set(s, l2.set(l1.get(s), b))`

Reverse derivative categories

- The potential applications of optics in scientific and statistical computing extend far beyond simple ADT manipulation
- eg. lenses can be used to implement **reverse-mode automatic differentiation**
- For a differentiable function $f : S \rightarrow A$, create a `Lens[S, A]` with `get(s) = f(s)` and `set(s', a) = \left( \frac{\partial f}{\partial s} \Big|_{s=s'} \right)^\top a`
- Then, for a composition of differentiable functions, lens composition implements the back-propagation chain rule

Forwards-backwards algorithms

- In the context of state space models, various kinds of forwards-backwards algorithms naturally arise
- For HMMs and DLMS, Bayesian inference for the hidden state can be computed with a forward pass for filtering, followed by a backward pass for smoothing (using the results of the forward pass)
- `get` can be used to compute a forward step, and `set` to implement a backward step
- A full algorithm can be implemented by composing the lenses corresponding to each step
- The associativity of lens composition can sometimes lead to algorithms that temporally parallelise

Conclusion

Summary

- All of the programming languages commonly used for Bayesian computation are poorly suited to the task
- Functional programming languages borrowing ideas from **Cartesian closed categories** provide an excellent foundation for scalable, compositional modelling and computation, including probabilistic and differentiable (array) programming
 - Concepts from category theory, such as **functors**, **monads**, **comonads**, and **lenses** provide a useful framework for organising computation
- This isn't about one particular programming language or language syntax — it is about a class of languages (statically typed compiled functional programming languages with type inference and higher-kinded types) and algorithmic approaches (eg. lazy transformation of immutable data structures with pure functions)

References

-  Elliott, C. (2017) **Generic functional parallel algorithms: Scan and FFT**. *ICFP 2017*, 1:7.
-  Heunen, C., Kammar, O., Staton, S., Yang, H. (2017) **A convenient category for higher-order probability theory**, *32nd ACM/IEEE LICS.*, 1–12.
-  Schauer, M., van der Meulen, F., Wang, A. Q. (2026) **Compositionality in algorithms for smoothing**, *Compositionality*, <https://doi.org/10.46298/compositionality-8-3>.
-  Särkkä, S., García-Fernández, A. F. (2019) **Temporal parallelization of Bayesian smoothers**, *arXiv*, 1905.13002.
-  Ścibior, A., Kammar, O., Gharamani, Z. (2018) **Functional programming for modular Bayesian inference**, *Proc. ACM Prog. Lang.*, 2(ICFP): 83.
-  Shiebler, D., Gavranovic, B., Wilson, P. (2021) **Category theory in machine learning**, *arXiv*, 2106.07032.
-  Milewski, B. (2026) **The dao of functional programming**, <https://github.com/BartoszMilewski/DaoFP/>.

[darrenjw.github.io](https://github.com/darrenjw)